

[75.07 / 95.02 / TB025]

Algoritmos y programación III

Paradigmas y programación

Trabajo práctico 2: Mafia

Nombre	Padrón	Mail
Joaquín Saladino	111220	jsaladino@fi.uba.ar
Nicolas Reiji Suyama	110789	nsuyama@fi.uba.ar
Ciriaco Lucas Leonel Vasquez	112435	cvasquez@fi.uba.ar
Sebastián Eduardo Yurtgulu	107498	syurtgulu@fi.uba.ar

Nota final:

Índice

Supuestos.....	3
Diagrama de clases.....	4
Diagramas de secuencia de una fase nocturna y diurna completa.....	5
Diagramas de paquetes.....	7
Detalles de implementación.....	7
CartaRol.....	7
Fases.....	8
Habilidad.....	9
Excepciones.....	9
JugadorSinCartaExcepcion.....	9
MazoVacioException.....	9
ObjetivoInvalidoException.....	9
RolNoVisibleException.....	9

Supuestos

- **Restricciones:**

El sistema valida mediante la clase **ConfiguracionPartida** que el número de jugadores se encuentre dentro del rango 5 a 12. Esta validación permite que se le pueda asignar una cantidad equilibrada de CartaRol. Generando un juego justo.

- **Aleatoriedad :**

Al iniciar la partida, el **Mazo** utiliza algoritmos de barajado sobre la colección de roles, asegurando que la distribución sea no determinista.

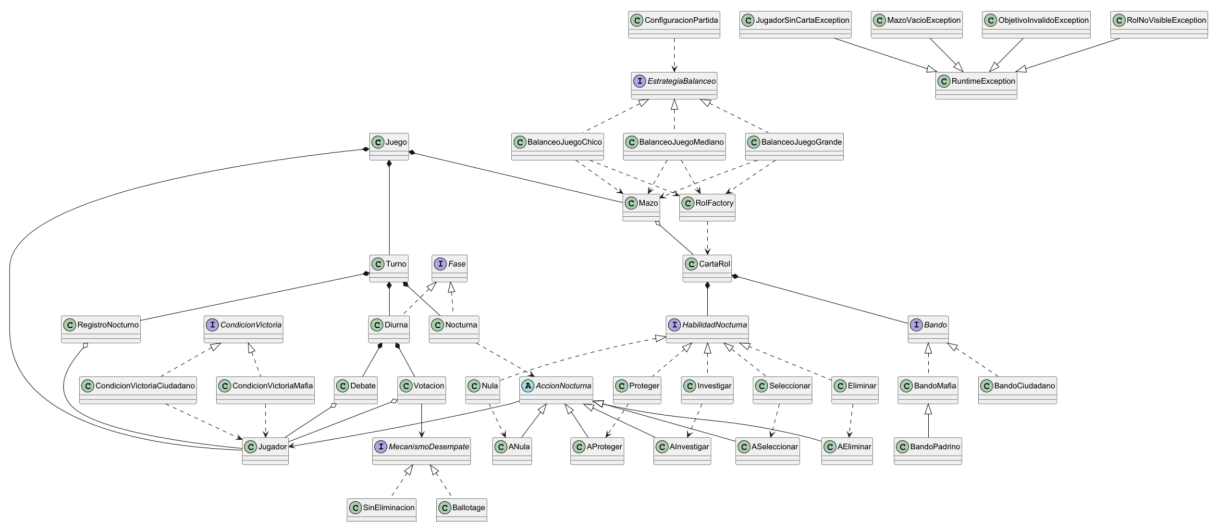
- **Entorno de Juego:**

Todos los jugadores comparten el mismo dispositivo. La interfaz de usuario implementa un flujo de "entrega de turno", para la ocultación de roles y acciones.

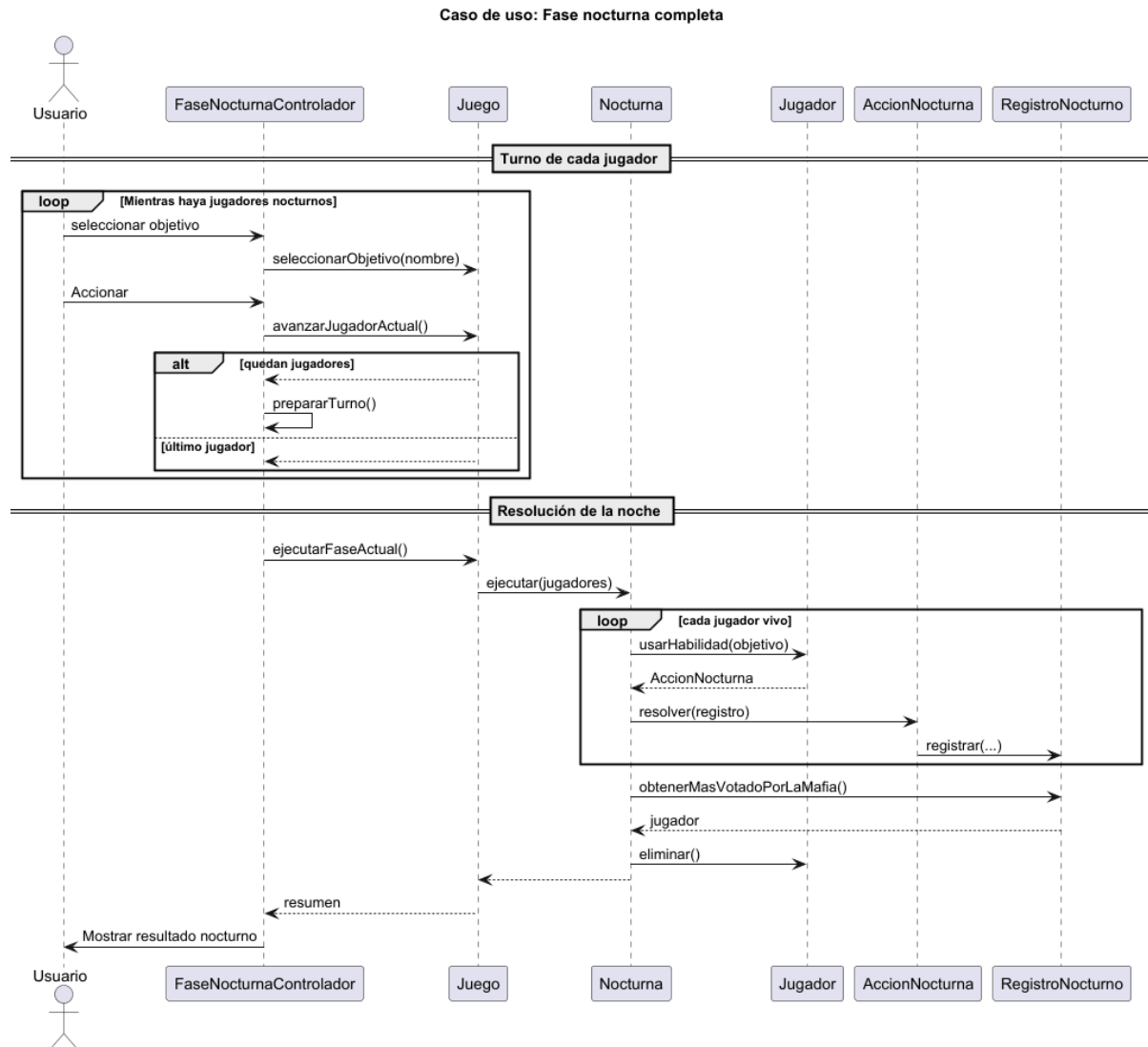
- **Revelación de Roles :**

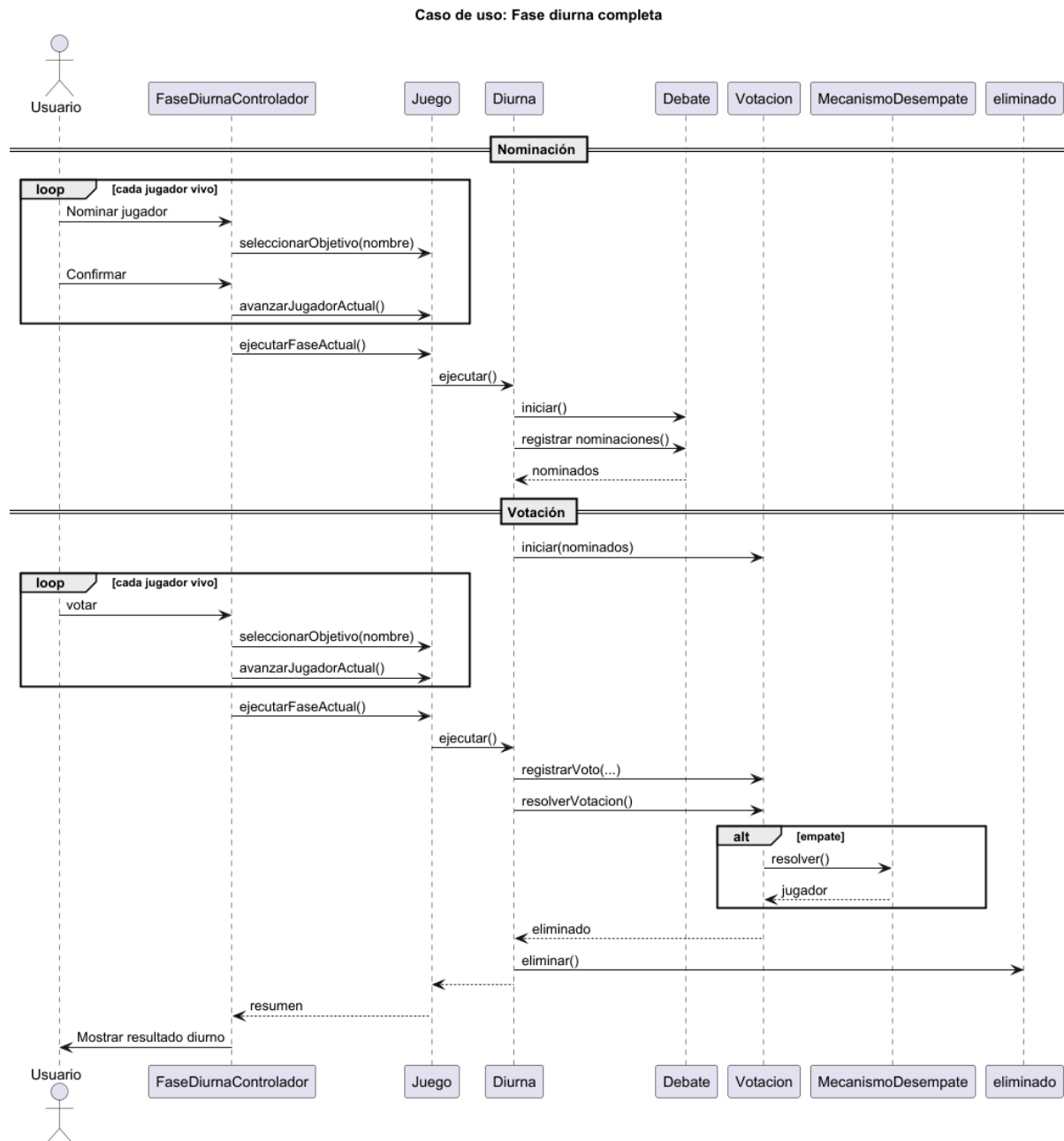
La revelación del rol del jugador se realiza únicamente en el momento en que su rol debe interactuar con la fase nocturna.

Diagrama de clases

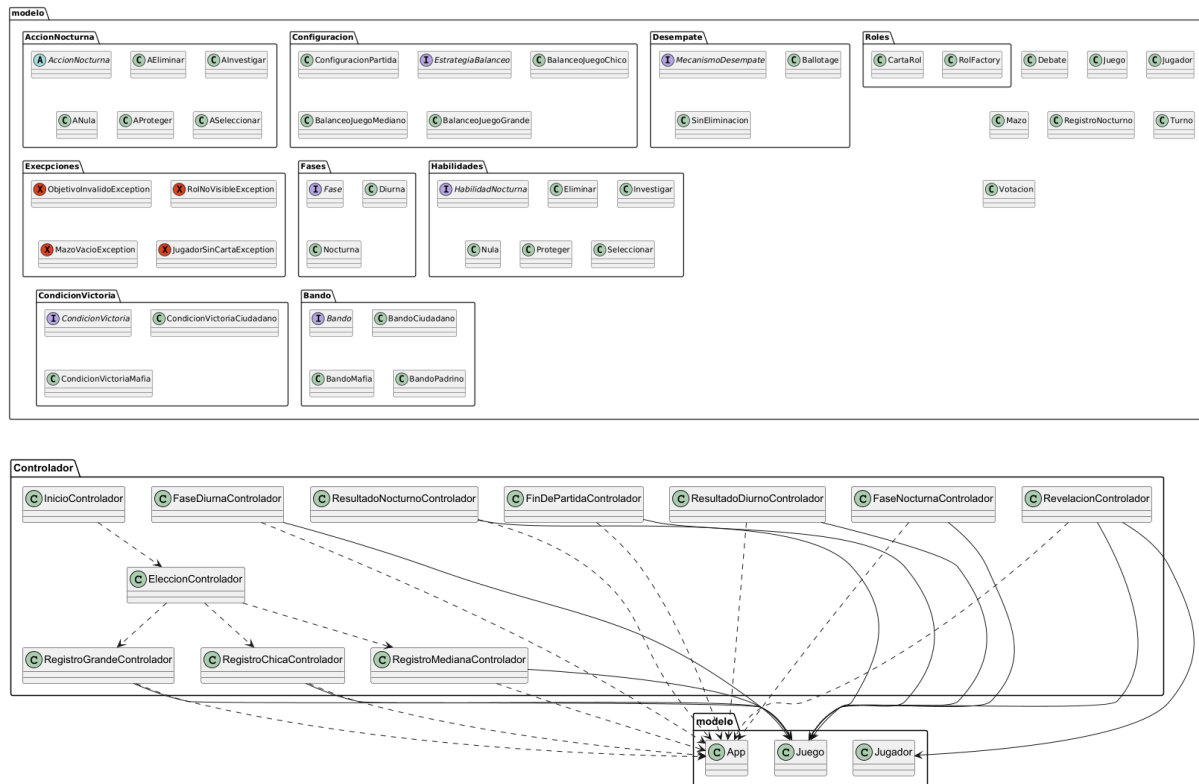


Diagramas de secuencia de una fase nocturna y diurna completa





Diagramas de paquetes



Detalles de implementación

CartaRol

La implementación de la clase **CartaRol** se apoya en tres patrones de diseño fundamentales que garantizan un sistema desacoplado, extensible y mantenible:

- **Patrón Strategy:** Se utilizó para encapsular las habilidades nocturnas mediante la interfaz **HabilidadNocturna**. Esto permite que el comportamiento de cada rol sea intercambiable sin modificar la clase **CartaRol**. Gracias a esta abstracción, los requisitos de acción y validación de objetivos se resuelven de forma polimórfica, facilitando la incorporación de nuevos roles.
- **Patrón Bridge:** Se aplicó para separar la identidad del rol de su afiliación a un bando. En lugar de crear una jerarquía compleja de clases (ej: **CiudadanoCiudadano**, **CiudadanoDetective**), definimos la interfaz

Bando. Esto desacopla la lógica de juego propia del rol de la lógica de pertenencia al grupo, permitiendo que un mismo comportamiento de juego pueda combinarse con distintos bandos si fuera necesario en el futuro.

- **Patrón Factory Method:** Se implementó para centralizar la creación de instancias de **CartaRol**. Este patrón evita la proliferación de subclases innecesarias para cada tipo de rol. A través de una clase factoría (**RolFactory**), es posible instanciar objetos **CartaRol** configurados con los parámetros (habilidad y bando) correctos, simplificando la gestión de la configuración inicial del juego y reduciendo la complejidad estructural del código.

Fases

La arquitectura del sistema de fases se fundamenta en la aplicación de los patrones **Strategy** y **State**, los cuales permiten gestionar la complejidad del ciclo de vida de la partida de forma modular y desacoplada.

- **Patrón Strategy:** Se implementa a través de la interfaz **Fase**, la cual define un contrato de métodos comunes (**iniciar**, **ejecutar**, etc). La clase **Turno** (contexto) interactúa con esta interfaz, delegando la lógica de negocio a las implementaciones concretas **Diurna** y **Nocturna**. Este enfoque elimina la necesidad de estructuras condicionales complejas (**if/else** o **switch**) dentro de **Turno** y facilita la incorporación de nuevas fases en el futuro sin modificar el motor principal.
- **Patrón State:** Se utiliza internamente en la clase **Diurna** para gestionar las sub-etapas de "Nominación" y "Votación". En lugar de implementar toda la lógica de control de flujo dentro de una única clase, el comportamiento de los métodos varía según el estado interno del objeto. Esto permite encapsular las reglas de negocio específicas de cada etapa y simplifica el mantenimiento del código, evitando que la clase **Diurna** crezca de forma desmedida y facilitando la transición ordenada entre los distintos estados del día.

Habilidad

La implementación de las habilidades nocturnas se basa en el **Patrón Command**, permitiendo transformar cada acción de juego en un objeto independiente.

- **Patrón Command** : Esta arquitectura encapsula la solicitud de ejecución (por ejemplo, "investigar al jugador X") dentro de un objeto **AccionNocturna**. Esto ofrece una ventaja crucial: la **ejecución diferida**. En lugar de procesar la acción en el mismo instante en que el jugador la elige, el sistema puede recolectar estas intenciones en una estructura de datos (el **RegistroNocturno**) para resolverlas todas de forma centralizada al finalizar la fase.

Excepciones

JugadorSinCartaExcepcion

Impide que un jugador sin carta asignada realice cualquier operación que dependa de su rol. Si esto ocurre es porque hay una falla en el flujo del setup del juego.

MazoVacioException

Evita repartir más cartas de las que hay en el mazo. Esta no es atrapada, si ocurre es por alguna modificación en ConfiguraciónPartida, ya que la cantidad de cartas es igual a la cantidad de jugadores.

ObjetivoInvalidoException

Rechaza la acción cuando el objetivo seleccionado no cumple con las condiciones para accionarse. Se atrapa en esObjetivoValido que devuelve false.

RolNoVisibleException

Garantiza la regla de juego de que las cartas no se revelen hasta que el jugador es eliminado o termina el juego. La lógica del juego impide que se vea el rol de una carta de un jugador vivo.